

7th EXERCISE: SUM OF DIGITS

WHAT'S THE PROBLEM?

In this exercise, we need to create a function that adds up all the individual digits in a whole number. For example, if we have the number 1234, our function should calculate $1 + 2 + 3 + 4 = 10$.

This might seem like a simple problem, but it's a great way to learn about loops, division, and how to break down a number into its individual parts.

REAL-WORLD EXAMPLE

Think about how you might count up the individual digits on a car number plate, or add up the digits of your phone number. For instance, if your phone number is 07123 456789, you might want to add up all those digits to get a single number (which equals 52).

HOW DO WE SOLVE THIS?

To add up the digits of a number, we need a way to:

1. Look at each digit separately
2. Add them together as we go

The key insight is that we can use **division and remainder** to extract the digits one by one.

THE MATHS BEHIND IT

In the decimal system (base 10), we can:

- Extract the last digit of a number by finding the remainder when dividing by 10
- Remove the last digit by performing integer division by 10

For example, with the number 1234:

- $1234 \div 10 = 123$ remainder 4 (so the last digit is 4)
- After integer division: $1234 \div 10 = 123$ (last digit removed)

Then we repeat with 123, and so on.

THE STEP-BY-STEP APPROACH

Here's how our algorithm works:

1. Start with a total of 0
2. Extract the last digit of the number using the remainder operation ($n \% 10$)
3. Add this digit to our running total
4. Remove the last digit by integer division ($n \div 10$)
5. Repeat steps 2-4 until we've processed all digits (when the number becomes 0)
6. Return the final total

LET'S SEE IT IN PYTHON

```
def sum_digits(n):
    total = 0
    while n > 0:
        total += n % 10 # Extract the last digit and add it to total
        n //= 10        # Remove the last digit
    return total

print(sum_digits(1234)) # Output: 10
```

In Python, the `//` operator performs integer division, giving us a whole number result.

C IMPLEMENTATION

```
#include <stdio.h>

int sum_digits(int n) {
    int total = 0;
    while (n > 0) {
        total += n % 10; # Add the last digit
        n /= 10;        # Remove the last digit
    }
    return total;
}

int main() {
    printf("%d\n", sum_digits(1234)); // Output: 10
    return 0;
}
```

BASH IMPLEMENTATION (for shell script fans)

```
sum_digits() {
    local n=$1
    local total=0
    while [ "$n" -gt 0 ]; do
        total=$((total + n % 10))
        n=$((n / 10))
    done
    echo "$total"
}

sum_digits 1234 # Output: 10
```

LET'S TRACE THROUGH AN EXAMPLE

Let's follow the algorithm step by step for the number 1234:

1. Start with `total = 0`
2. First iteration with `n = 1234`:
 - Get last digit: $1234 \% 10 = 4$
 - Add to total: $0 + 4 = 4$
 - Remove last digit: $1234 \div 10 = 123$
3. Second iteration with `n = 123`:
 - Get last digit: $123 \% 10 = 3$
 - Add to total: $4 + 3 = 7$
 - Remove last digit: $123 \div 10 = 12$
4. Third iteration with `n = 12`:
 - Get last digit: $12 \% 10 = 2$
 - Add to total: $7 + 2 = 9$
 - Remove last digit: $12 \div 10 = 1$
5. Fourth iteration with `n = 1`:
 - Get last digit: $1 \% 10 = 1$
 - Add to total: $9 + 1 = 10$
 - Remove last digit: $1 \div 10 = 0$

6. Now $n = 0$, so we exit the loop
7. Return `total = 10`

A VISUAL STEP-BY-STEP

Here's a table showing how the values change with each step:

Step	Number (n)	Last Digit (n % 10)	Running Total
Start	1234	-	0
1	123	4	$0 + 4 = 4$
2	12	3	$4 + 3 = 7$
3	1	2	$7 + 2 = 9$
4	0	1	$9 + 1 = 10$
End	0	-	10

HOW EFFICIENT IS THIS SOLUTION?

This algorithm is very efficient:

- **Time Efficiency:** It takes time proportional to the number of digits in the number. A number N has approximately $\log_{10}(N)$ digits, so we say the time complexity is $O(\log N)$.
- **Memory Efficiency:** It uses a fixed amount of memory regardless of how big the input number is (just two variables: the input number and the running total). We call this $O(1)$ space complexity.

COOL PROPERTIES AND APPLICATIONS

The sum of digits has some interesting properties:

1. **Digital Root:** If you keep summing the digits until you get a single digit, you find what's called the "digital root". For example, the digital root of 1234 is 1, because $1+2+3+4=10$, and $1+0=1$.
2. **Divisibility Rule:** A number is divisible by 9 if and only if the sum of its digits is divisible by 9. For example, 1234 has a digit sum of 10, which is not divisible by 9, so 1234 is not divisible by 9.
3. **Check Digits:** Some identification numbers (like ISBNs) use check digits calculated using operations similar to digit sums to verify the validity of the number.

This concept is used in:

- Data validation (checking credit card numbers)
- Number theory in mathematics
- Some cryptography algorithms
- Various puzzles and recreational mathematics

MAKING IT EVEN BETTER

We can extend our solution in several ways:

1. Handling negative numbers

```
def sum_digits(n):  
    n = abs(n) # Take absolute value to handle negative numbers  
    total = 0  
    while n > 0:  
        total += n % 10  
        n //= 10  
    return total
```

2. Different number bases

If you wanted to sum the digits of a binary or hexadecimal number:

```
def sum_digits_base(n, base=10):
    total = 0
    while n > 0:
        total += n % base
        n //= base
    return total

# Example: Sum digits of 1234 in base 10
print(sum_digits_base(1234, 10)) # Output: 10

# Example: Sum digits of binary 1010 (which is 10 in decimal)
print(sum_digits_base(10, 2)) # Output: 2 (1+0+1+0)
```

3. Finding the digital root

```
def digital_root(n):
    # Keep summing digits until we get a single-digit result
    while n >= 10: # While n has more than one digit
        n = sum_digits(n)
    return n

print(digital_root(1234)) # Output: 1
```

ALTERNATIVE APPROACHES

There are other ways to solve this problem:

String conversion method

A common alternative is to convert the number to a string and sum its individual characters:

```
def sum_digits_string(n):
    # Convert to string, then convert each character back to int and sum
    return sum(int(digit) for digit in str(n))
```

This is concise but uses more memory, as we're creating a new string.

WHAT WE'VE LEARNED

This exercise teaches us:

1. **Breaking Down Numbers:** How to extract and work with individual digits
2. **Loop Design:** Creating a loop that processes data step by step
3. **Memory Efficiency:** Solving a problem with minimal memory usage
4. **Algorithm Tracing:** Following the execution of an algorithm step by step
5. **Mathematical Connections:** Connecting programming to number theory

The sum of digits problem might seem simple at first, but it introduces important programming concepts and has surprising connections to mathematics and real-world applications.